

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC FPT**

**KIẾN TRÚC NHẬN THỨC HAI TẦNG CHO PHÁT HIỆN
VÀ PHẢN HỒI MỖI ĐE DỌA TỰ ĐỘNG SỬ DỤNG AI
TÁC TỬ (AGENTIC AI)**

thực hiện bởi

Nguyễn Đức Bình

Luận văn được nộp để đáp ứng yêu cầu
của học vị Thạc sĩ Kỹ thuật Phần mềm

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC FPT**

**KIẾN TRÚC NHẬN THỨC HAI TẦNG CHO PHÁT HIỆN
VÀ PHẢN HỒI MỐI ĐE DỌA TỰ ĐỘNG SỬ DỤNG AI
TÁC TỬ (AGENTIC AI)**

thực hiện bởi

Nguyễn Đức Bình

Luận văn được nộp để đáp ứng yêu cầu
của học vị Thạc sĩ Kỹ thuật Phần mềm

Giảng viên hướng dẫn:

TS. Bùi Văn Hiệu

TS. Đặng Văn Hiếu

Tóm tắt

Trong bối cảnh an ninh mạng hiện đại, các Trung tâm Điều hành An ninh (SOC) đối mặt với tình trạng quá tải cảnh báo khi khối lượng nhật ký liên tục vượt xa khả năng xử lý của chuyên viên phân tích. Các công cụ tự động hoá truyền thống vận hành theo kịch bản tĩnh thiếu khả năng suy luận theo ngữ cảnh, trong khi đưa trực tiếp nhật ký thô vào Mô hình Ngôn ngữ Lớn (LLM) lại vấp phải nút thắt độ trễ, nguy cơ tiêm nhiễm câu lệnh và vi phạm chủ quyền dữ liệu. Luận văn đề xuất SENTINEL, một kiến trúc an ninh nhận thức hai tầng vận hành cục bộ hoàn toàn (air-gapped), kết hợp tác tử suy luận LLM với Sinh có Tăng cường Truy xuất (RAG). Tầng 1 vận hành như một bộ lọc tốc độ đường truyền—luật xác định, chấm bắt thường Welford và cổng LightGBM—để tự động loại bỏ nhiễu nền của luồng, trong khi Tầng 2 sử dụng tác tử AI suy luận có trạng thái, dựa trên tri thức MITRE ATT&CK và hướng dẫn NIST, để xử lý các đe dọa phức tạp phía sau các rào chắn mật mã. Kết quả thực nghiệm trên các bộ dữ liệu xâm nhập chuẩn cho thấy hiệu quả của SENTINEL. Tầng 1 cùng Cổng Học máy tự động gỡ tải **90,6%** lưu lượng đầu vào. Vì sự kiện điển hình không bao giờ chạm tới mô hình, độ trễ toàn tuyến ở mức trung vị giảm còn **0,88 ms** so với **17.174,7 ms** của đường LLM đơn tầng. Cơ chế đóng gói Nonce vô hiệu hoá tiêm nhiễm câu lệnh bằng cấu trúc chứ không bằng nhận dạng nội dung, kháng được toàn bộ **678** mẫu đối kháng mà lớp lọc tĩnh không hấp thụ được; chuỗi niêm phong HMAC-SHA256 phát hiện mọi kiểu giả mạo cốt lõi, qua đó bảo đảm tính toàn vẹn và khả năng phát hiện giả mạo của nhật ký kiểm toán—song do dùng khoá đối xứng, không cấu thành chống chối bỏ theo nghĩa mật mã học—toàn bộ vận hành trên một máy trạm local. SENTINEL chứng minh kiến trúc lai hai tầng là một mô hình phòng thủ đáng tin cậy và hiệu quả cho SOC hiện đại.

Lời cảm ơn

Để hoàn thành công trình nghiên cứu này và đạt được học vị Thạc sĩ Kỹ thuật Phần mềm, tôi đã nhận được sự hỗ trợ, động viên và dìu dắt quý báu từ rất nhiều cá nhân và tập thể.

Trước hết, tôi xin bày tỏ lòng biết ơn chân thành và kính trọng sâu sắc nhất tới hai thầy hướng dẫn khoa học của tôi, TS. Bùi Văn Hiệu và TS. Đặng Văn Hiếu. Với tri thức học thuật uyên bác, sự định hướng chiến lược chuẩn xác và tinh thần làm việc nghiêm túc, quý thầy đã luôn dành thời gian kiên nhẫn chỉ bảo, định hướng tư duy phản biện và truyền cảm hứng nghiên cứu cho tôi trong suốt chặng đường thực hiện luận văn. Những góp ý chuyên môn sắc bén của quý thầy không chỉ giúp tôi hoàn thiện công trình này mà còn là hành trang tri thức vô giá cho sự nghiệp nghiên cứu an toàn thông tin và trí tuệ nhân tạo của tôi sau này.

Tôi xin trân trọng gửi lời tri ân tới Ban Giám hiệu, Ban Lãnh đạo Viện Quản trị & Công nghệ FSB cùng toàn thể quý thầy cô Trường Đại học FPT đã truyền dạy nền tảng tri thức lý thuyết và thực tiễn vững chắc trong một môi trường nghiên cứu chuyên nghiệp, hiện đại. Tôi cũng xin chân thành cảm ơn Ban Lãnh đạo cùng các đồng nghiệp tại Cục C12, Trung tâm Dữ liệu Quốc gia—sự tin tưởng, điều kiện thuận lợi về thời gian cùng những kinh nghiệm thực chiến phong phú từ đơn vị đã đem lại góc nhìn thực tiễn sâu sắc, áp dụng trực tiếp vào quá trình thiết kế, thực nghiệm và kiểm chứng hệ thống SENTINEL.

Cuối cùng, và cũng là niềm tự hào lớn lao nhất, tôi xin dành trọn lòng tri ân sâu sắc nhất tới gia đình thân yêu, đặc biệt là người vợ hiền cùng con nhỏ. Sự hy sinh thầm lặng, lòng kiên nhẫn vô bờ bến và nguồn động viên không ngừng nghỉ của vợ và con trong những tháng ngày tôi miệt mài với các đợt thực nghiệm kéo dài xuyên đêm chính là điểm tựa vững chắc nhất giúp tôi hoàn thành công trình này. Mặc dù đã rất cố gắng, luận văn khó tránh khỏi hạn chế; tôi rất mong nhận được sự chỉ dẫn và đóng góp quý báu từ quý thầy cô trong Hội đồng chấm luận văn cùng các đồng nghiệp.

Mục lục

Tóm tắt	i
Lời cảm ơn	ii
Danh sách hình vẽ	v
Danh sách bảng	v
Danh mục các từ viết tắt	vi
1 Giới thiệu	1
1.1 Bối cảnh và Động lực Nghiên cứu	1
1.2 Mục tiêu Nghiên cứu	2
1.3 Câu hỏi Nghiên cứu	3
1.4 Phạm vi và Đối tượng Nghiên cứu	4
1.5 Phương pháp Nghiên cứu	5
1.6 Đóng góp Cốt lõi của Luận văn	5
1.7 Cấu trúc của Luận văn	6
2 Cơ sở Lý thuyết và Tổng quan Nghiên cứu	7
2.1 Thực trạng Vận hành SOC và Nút thắt Nhận thức	7
2.2 Lý thuyết Thống kê Trực tuyến và Học máy Tầng 1	7
2.3 Mô hình Ngôn ngữ Lớn Cục bộ và Trí tuệ Tác tử Tầng 2	8
2.4 An ninh Đối kháng AI và Mật mã học Kiểm toán Nhật ký	9
2.5 Tổng quan Nghiên cứu Liên quan và Khoảng trống Nghiên cứu	9
3 Thiết kế Hệ thống và Triển khai Kỹ thuật	11
3.1 Kiến trúc Tổng thể SENTINEL và Tích hợp Ngăn xếp SOC	11
3.2 Tầng 1: Bộ lọc Tốc độ Đường truyền và Cổng Học máy	13
3.3 Tầng 2: Tác tử Nhận thức LangGraph và Bộ nhớ Đe dọa	14
3.4 Cơ chế Truy xuất Tri thức Lai Dual-RAG	15
3.5 Kiến trúc An ninh AI và Niêm phong Kiểm toán Mật mã	15
3.6 SOC Dashboard và Kiến trúc Triển khai Cục bộ	16
4 Thực nghiệm và Đánh giá	17
4.1 Môi trường Thực nghiệm và Giao ước Đo lường	17
4.2 RQ1 — Xả tải và Kinh tế học Tầng lọc	18
4.3 RQ2 — Rào chắn An ninh AI và Toàn vẹn Pháp y	21
4.4 RQ3 — Suy luận Có trạng thái và Quy kết Kỹ thuật	25

5	Kết luận và Hướng phát triển	27
5.1	Tổng hợp Kết quả Nghiên cứu	27
5.2	Đóng góp Khoa học và Thực tiễn	29
5.3	Giới hạn và Hướng Phát triển	29
	Tài liệu tham khảo	1

Danh sách hình vẽ

Hình 1.	Luồng thực thi vận hành trong pipeline SENTINEL. (Nguồn: Tác giả)	12
---------	---	----

Danh sách bảng

Bảng 1.	Bảng so sánh đối chiếu giữa SENTINEL và các giải pháp phòng thủ hiện hành	10
Bảng 2.	Cấu hình Thực nghiệm và Phân vai Tập Dữ liệu	18
Bảng 3.	Tỉ lệ Xả tải theo Từng Tầng trên Hai Luồng có Base-rate Khác nhau	18
Bảng 4.	Thông lượng các cấu hình đối chứng trên cùng mẫu con 150 sự kiện phân tầng	20
Bảng 5.	Các lớp rào chắn, mẫu số phép đo và kết quả	22

Danh mục các từ viết tắt

Từ viết tắt	Định nghĩa đầy đủ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
APT	Advanced Persistent Threat (Mối đe dọa thường trực nâng cao)
CTI	Cyber Threat Intelligence (Tình báo mối đe dọa mạng)
DAPT	Dynamic Advanced Persistent Threat (Tấn công APT động)
FAISS	Facebook AI Similarity Search (Cơ máy tìm kiếm tương đồng FAISS)
FPR	False Positive Rate (Tỷ lệ báo động giả)
GGUF	GPT-Generated Unified Format (Định dạng hợp nhất GGUF)
HITL	Human-in-the-Loop (Con người trong vòng lặp)
HMAC	Hash-based Message Authentication Code (Mã xác thực thông điệp dựa trên hàm băm)
LLM	Large Language Model (Mô hình ngôn ngữ lớn)
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge (Khung tri thức chiến thuật và kỹ thuật tấn công MITRE)
NIST	National Institute of Standards and Technology (Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ)
RAG	Retrieval-Augmented Generation (Sinh văn bản tăng cường truy xuất)
RRF	Reciprocal Rank Fusion (Tổng hợp xếp hạng nghịch đảo)
SIEM	Security Information and Event Management (Quản lý thông tin và sự kiện an ninh)
SOAR	Security Orchestration, Automation, and Response (Điều phối, tự động hóa và phản hồi an ninh)
SOC	Security Operations Center (Trung tâm điều hành an ninh mạng)

Chương 1

Giới thiệu

Chương này trình bày bối cảnh nghiên cứu, bài toán trung tâm và cơ sở thiết kế hệ thống SENTINEL—một kiến trúc nhận thức hai tầng tự động hóa quy trình phát hiện và phản hồi sự cố trong các Trung tâm Điều hành An ninh (SOC). Bằng việc phân tích hạn chế của phương pháp phòng thủ tắt định truyền thống, tình trạng quá tải cảnh báo, cùng rào cản độ trễ và lỗi đồng bộ kháng khi ứng dụng Mô hình Ngôn ngữ Lớn (LLM), chương này xác lập mục tiêu, câu hỏi nghiên cứu, phạm vi, phương pháp luận và các đóng góp cốt lõi của luận văn.

1.1 Bối cảnh và Động lực Nghiên cứu

Một Trung tâm Điều hành An ninh (SOC) hiện đại mỗi ngày nhận về hàng triệu sự kiện nhật ký [2], trong đó phần áp đảo là hoạt động bình thường. Vì sợ bỏ lọt, các hệ thống giám sát thường được đặt ở ngưỡng rất nhạy, nên số cảnh báo sinh ra càng nhiều và tỉ lệ báo động giả có thể vượt 80% [4]. Hệ quả là chỗ nghẽn của một SOC hiếm khi nằm ở khả năng phát hiện. Nó nằm ở khâu tiếp theo: quyết định xem cảnh báo nào đáng để một chuyên gia dừng lại xem xét. Sự chú ý của con người mới là nguồn lực khan hiếm, và nó không nhân lên được bằng cách mua thêm máy chủ.

Các công cụ tự động hoá đang dùng chưa gánh được khâu đó. Thiết bị lọc theo luật tĩnh và chữ ký cố định chỉ nhận ra những gì đã được mô tả trước, nên mọi tình huống nằm ngoài mô tả đều bị đẩy sang cho người [5]. Chúng phân loại được, nhưng không giải thích được vì sao; mà việc chuyên gia phải làm lại chính là hiểu bối cảnh của một cảnh báo trước khi quyết định.

Mô hình Ngôn ngữ Lớn (LLM) hứa hẹn đúng năng lực còn thiếu ấy: đọc một sự kiện trong bối cảnh của nó và diễn giải bằng ngôn ngữ con người hiểu được. Nhưng đưa một mô hình như vậy vào giữa luồng dữ liệu vận hành thì sinh ra ba khoản chi phí mới. Thứ nhất là chi phí thời gian và tài nguyên: mỗi lần suy luận mất vài giây, trong khi dữ liệu đến liên tục [18]. Thứ hai là chi phí niềm tin: bản thân tầng suy luận trở thành một mục tiêu, bởi nhật ký có thể mang theo câu chữ do kẻ tấn công cố ý soạn để lái mô hình [9], và những gì mô hình kết luận thì phải chứng minh được là chưa bị sửa đổi khi đưa ra làm chứng cứ. Thứ ba là chi phí chủ quyền dữ liệu: nhật ký nội bộ thường không được phép rời khỏi tổ chức, nên mô hình phải chạy tại chỗ trên phần cứng hữu hạn [31].

Quan sát nền tảng dẫn tới thiết kế của luận văn là chênh lệch chi phí giữa hai loại phán quyết. Một phán quyết bằng luật thống kê tốn chưa tới một phần nghìn giây; một phán quyết bằng mô hình ngôn ngữ tốn hàng giây. Khoảng cách bốn đến năm bậc độ lớn ấy khiến việc đem mọi sự kiện qua tầng đắt nhất trở thành lãng phí có hệ thống, trong khi tuyệt đại đa số sự kiện vốn có thể kết luận được bằng những phép kiểm rất rẻ. Nghiên cứu vì thế đề xuất SENTINEL, một kiến trúc hai tầng theo nguyên lý đặt phán quyết rẻ trước phán quyết đắt: một tầng lọc nhanh giải quyết dứt điểm phần lớn lưu lượng, và một tầng suy luận chỉ được gọi tới cho phần còn lại, phía sau các rào chắn bảo vệ và một vết kiểm toán niêm phong.

Ba khoản chi phí nêu trên định hình ba hướng nghiên cứu của luận văn, và điều làm chúng trở thành bài toán nghiên cứu chứ không chỉ là bài toán kỹ thuật là ở chỗ cả ba đều phải đo được, trên dữ liệu thật, bằng những thước đo không tự tăng bốc.

1.2 Mục tiêu Nghiên cứu

Mục tiêu tổng quát của luận văn là nghiên cứu thiết kế, xây dựng nguyên mẫu và đánh giá thực nghiệm kiến trúc SENTINEL nhằm tự động hóa quy trình phát hiện, suy luận ngữ cảnh và phản hồi sự cố an ninh mạng trên hạ tầng cục bộ, đạt tối ưu về độ trễ xử lý và đảm bảo an toàn mật mã.

Mục tiêu tổng quát trên được cụ thể hóa thông qua ba nhiệm vụ nghiên cứu trọng tâm, đặt tương ứng một-một với ba câu hỏi nghiên cứu ở Mục 1.3:

Thứ nhất, xây dựng đường ống lọc hai chặng phi trạng thái tại Tầng 1 vận hành ở tốc độ đường truyền, kết hợp thuật toán thống kê Welford $O(1)$, Cổng Học máy LightGBM và bộ đệm Semantic Cache Tầng 1.75 nhằm giải quyết dứt điểm phần lớn lưu lượng ngay tại tầng rẻ, không tiêu tốn tài nguyên GPU.

Thứ hai, thiết kế cơ chế bảo vệ tất định cho mô hình ngôn ngữ Tầng 2 thông qua kỹ thuật Đóng gói Dữ liệu Phân định sử dụng Nonce mật mã ngẫu nhiên, rào chắn Guardrail kiểm soát danh mục hành động đầu ra và chuỗi niêm phong HMAC-SHA256 nhằm vô hiệu hóa các cuộc tấn công tiêm nhiễm câu lệnh và bảo đảm tính toàn vẹn, phát hiện giả mạo đối với vết pháp y.

Thứ ba, phát triển cỗ máy suy luận có trạng thái dựa trên đồ thị tác tử LangGraph kết hợp Bộ nhớ Đề dọa bền vững và hệ thống RAG kép (FAISS và BM25) tích hợp tri thức MITRE ATT&CK chuẩn STIX, cho phép sinh phán quyết có trích dẫn bằng chứng, phát hiện và vô hiệu hoá hư cấu bằng rào chắn neo-RAG tất định, đồng thời phân loại chính xác phần công việc bắt buộc phải chuyển cho chuyên gia.

1.3 Câu hỏi Nghiên cứu

Tiến trình thiết kế và đánh giá thực nghiệm của luận văn được dẫn dắt bởi ba câu hỏi nghiên cứu chiến lược:

Câu hỏi nghiên cứu thứ nhất (RQ1) về hiệu năng đường ống và kinh tế học tầng lọc: Làm thế nào để thiết kế một kiến trúc phân tầng có khả năng xả tải tự động luồng dữ liệu an ninh ở tốc độ đường truyền nhằm giải quyết vấn đề “nút thắt cổ chai” về độ trễ và tiêu thụ tài nguyên của các Mô hình Ngôn ngữ Lớn?

Câu hỏi nghiên cứu thứ hai (RQ2) về rào chắn an ninh AI và tính toàn vẹn vết pháp y: Phương pháp luận và cơ chế bảo mật nào có khả năng chống lại các rủi ro đối kháng (điển hình như tiêm nhiễm prompt qua nhật ký) nhằm bảo vệ luồng suy luận của AI, đồng thời

bảo đảm tính toàn vẹn và khả năng phát hiện giả mạo của các vết chứng cứ pháp y?

Câu hỏi nghiên cứu thứ ba (RQ3) về suy luận tác tử có trạng thái và quy kết kỹ thuật: Làm thế nào để tích hợp năng lực suy luận có trạng thái và khả năng tra cứu tri thức chuyên ngành vào Tác tử AI nhằm tự động hóa quy trình phân tích, quy kết kỹ thuật tấn công và sinh ra các báo cáo pháp y minh bạch, qua đó giảm tải và phân loại chính xác phần công việc cần đến chuyên gia?

1.4 Phạm vi và Đối tượng Nghiên cứu

Đối tượng nghiên cứu trọng tâm của luận văn là Kiến trúc Nhận thức Hai tầng SENTINEL, bao gồm bộ lọc phi trạng thái Tầng 1 (thuật toán thống kê trực tuyến Welford và Cổng Học máy LightGBM), Tầng 1.75 Semantic Cache, cỗ máy tác tử có trạng thái Tầng 2 (đồ thị LangGraph và mô hình ngôn ngữ Foundation-Sec-8B-Instruct lượng tử hóa Q4_K_M vận hành qua llama.cpp), hệ thống RAG kép (kết hợp FAISS và BM25) tích hợp tri thức MITRE ATT&CK, cùng các cơ chế an ninh mật mã gồm Đóng gói Dữ liệu Phân định Nonce và chuỗi niêm phong kiểm toán HMAC-SHA256.

Phạm vi đánh giá dựa trên **hai tập dữ liệu chính**: CSE-CIC-IDS2018 cho luồng giám sát mạng diện rộng và CSIC 2010 cho tấn công HTTP Tầng 7 thực tế. Hai nguồn bổ trợ—DAPT2020 cho chuỗi xâm nhập đa giai đoạn và tập zero-day—chỉ đóng vai trò **minh chứng khả thi (proof-of-concept)**, không tham gia bất kỳ tuyên bố đóng góp nào. Đánh giá đối kháng dùng 703 mẫu từ ba bộ công bố (AdvBench, deepset, jackhhao) làm tỉ lệ chính, kèm 80 mẫu tác giả biên soạn báo cáo tách bạch. Hệ thống được triển khai trên một máy trạm cục bộ (NVIDIA RTX 4060 Ti 16GB VRAM, Intel Core i7, 32GB RAM), vận hành hoàn toàn cách ly mạng. Danh mục hành động tự động giới hạn ở chặn địa chỉ IP, ghi nhật ký hoặc chuyển chuyên gia duyệt; can thiệp ngắt kết nối hạ tầng vật lý nằm ngoài phạm vi.

1.5 Phương pháp Nghiên cứu

Luận văn áp dụng phương pháp Nghiên cứu Khoa học Thiết kế (DSR) trên ba bình diện: thống kê và học máy (thuật toán Welford trực tuyến $O(1)$ tính Z -score kết hợp mô hình phân loại LightGBM); logic tác tử (đồ thị trạng thái LangGraph điều phối suy luận ngôn ngữ, kết hợp truy xuất lai FAISS, BM25 và hợp nhất xếp hạng RRF); và bảo mật đối kháng (thử nghiệm ứng suất chống tiêm nhiễm câu lệnh và kiểm tra toàn vẹn vết nhật ký). Hiệu quả kiến trúc được đánh giá qua Khung chỉ số 5 chiều: chất lượng phân loại, hiệu năng xử lý, khả năng kháng đối kháng, độ nhất quán của căn cứ giải thích do mô hình độc lập chấm, và toàn vẹn vết nhật ký. Ba nguyên tắc phương pháp luận được tuân thủ xuyên suốt: (i) **mẫu số phải khớp câu hỏi**—chỉ số phụ thuộc phân bố lấy từ luồng thực, chất lượng phân loại lấy từ tập cân bằng lớp; (ii) dùng thước đo miễn nhiễm lệch lớp (MCC, Balanced Accuracy) thay vì Accuracy hay $F1$ nhị phân; (iii) mọi tỉ lệ đều báo kèm mẫu số và độ phủ phép đo.

1.6 Đóng góp Cốt lõi của Luận văn

Luận văn đóng góp ba giá trị cốt lõi vào lĩnh vực an toàn thông tin và ứng dụng trí tuệ nhân tạo.

Về kiến trúc hệ thống, nghiên cứu đề xuất và định lượng nguyên lý *đặt phán quyết rẽ trước phán quyết đất*, đồng thời chứng minh bằng số đo rằng tỉ lệ xả tải là **hàm của tỉ lệ tấn công trong luồng**, không phải hằng số của hệ—đặc tính chưa được các công trình liên quan phát biểu tường minh.

Về bảo mật AI, nghiên cứu đóng góp cơ chế phòng thủ mang tính cấu trúc dựa trên Đóng gói Dữ liệu Phân định với Nonce mật mã ngẫu nhiên. Điểm cốt lõi là bảo chứng này **không phụ thuộc vào tỉ lệ chặn của bất kỳ bộ phân loại nào**: dữ liệu trong vùng bọc được xử lý như văn bản thụ động bất kể nội dung, khác hẳn các bộ lọc ngữ nghĩa xác suất vốn suy giảm trước biến thể mới.

Về tác tử và tri thức bảo mật, nghiên cứu thay thế kịch bản SOAR cứng nhắc bằng đồ thị tác tử có trạng thái LangGraph trên kho tri thức 433 mục MITRE ATT&CK chuẩn STIX và Bộ nhớ Đe dọa bền vững, kèm rào chắn neo-RAG tắt định buộc mọi mã kỹ thuật công bố phải hiện diện trong tài liệu truy xuất của chính lô đó—biến việc chống hư cấu từ kỳ vọng vào mô hình thành ràng buộc kiểm chứng được của kiến trúc.

1.7 Cấu trúc của Luận văn

Chương 2 hệ thống hóa cơ sở lý thuyết và tổng quan công trình liên quan nhằm xác định khoảng trống nghiên cứu. Chương 3 trình bày thiết kế kỹ thuật của SENTINEL. Chương 4 trình bày kết quả thực nghiệm định lượng, tổ chức theo đúng ba câu hỏi nghiên cứu. Chương 5 tổng kết kết quả, thừa nhận giới hạn và phác thảo hướng phát triển.

Chương 2

Cơ sở Lý thuyết và Tổng quan Nghiên cứu

Chương này trình bày cơ sở lý thuyết nền tảng và tổng quan nghiên cứu liên quan: nút thắt vận hành SOC, cơ sở thống kê và học máy Tầng 1, kiến trúc tác tử và ngôn ngữ lớn Tầng 2, nguy cơ đối kháng AI cùng kỹ thuật kiểm toán mật mã, và khảo sát các hệ thống hiện hành nhằm xác định khoảng trống nghiên cứu mà SENTINEL giải quyết.

2.1 Thực trạng Vận hành SOC và Nút thắt Nhận thức

Công tác giám sát an toàn thông tin tại các SOC dựa trên việc thu thập nhật ký từ NGFW, IDS/IPS và EDR về hệ thống SIEM tập trung [2], ở khối lượng vượt quá khả năng phân tích thủ công [3]. Do các thiết bị phòng thủ ranh giới vận hành trên chữ ký tĩnh và tập luật tắt định, việc nói lỏng ngưỡng cảnh báo để tránh bỏ lọt đã dẫn tới quá tải thông báo với tỷ lệ báo động giả vượt 80% [4] và kiệt sức nhận thức cho chuyên gia phân tích. Giải pháp SOAR tuy tự động hóa phản hồi nhưng phụ thuộc vào playbook lập trình sẵn, thiếu khả năng suy luận ngữ cảnh trước biến thể tấn công mới [5].

2.2 Lý thuyết Thống kê Trực tuyến và Học máy Tầng 1

Để lọc luồng dữ liệu ở tốc độ đường truyền mà không lưu trữ toàn bộ lịch sử trong bộ nhớ, thuật toán thống kê trực tuyến Welford đạt độ phức tạp $\mathcal{O}(1)$ về thời gian và không gian [6]. Giá trị trung bình μ_k và tổng bình phương độ lệch S_k được cập nhật theo công thức truy hồi:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k}, \quad S_k = S_{k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \quad (2.1)$$

cho phép tính toán độ lệch chuẩn $\sigma_k = \sqrt{S_k/(k-1)}$ và Z -score dị thường liên tục mà không cần lưu chuỗi sự kiện thô. Song song với đó, mô hình LightGBM với cơ chế phát triển cây theo lá (leaf-wise) và phân nhóm đặc trưng dạng biểu đồ tần suất đạt tốc độ suy luận dưới một phần ba mili-giây trên mỗi mẫu. Để chống kỹ thuật né tránh học máy (anti-evasion), Cổng Học máy thực hiện kẹp ngưỡng Z -score trong khoảng $\pm 8\sigma$ và điền giá trị thiếu bằng trung bình đặc trưng. Ngoài ra, cơ chế Semantic Cache Tầng 1.75 sử dụng kỹ thuật băm chuỗi truy vấn chuẩn hoá trong RAM giúp tái sử dụng phán quyết cho các sự kiện sinh cùng một mô tả, ngắt luồng xử lý trước khi kích hoạt GPU.

2.3 Mô hình Ngôn ngữ Lớn Cục bộ và Trí tuệ Tác tử Tầng 2

Triển khai LLM cục bộ đòi hỏi tối ưu hóa bộ nhớ nghiêm ngặt: mô hình Foundation-Sec-8B-Instruct ở định dạng FP16 chiếm xấp xỉ 18 GB VRAM [23]. Kỹ thuật lượng tử hóa ánh xạ trọng số thực sang lưới số nguyên bit thấp [19]. Thông qua định dạng GGUF Q4_K_M trên llama.cpp, bộ nhớ VRAM yêu cầu giảm xuống 7–8 GB (>50%) mà không làm giảm đáng kể độ chính xác, vận hành trơn tru trên card GPU 16 GB. Để điều phối quy trình suy luận, Tầng 2 sử dụng đồ thị trạng thái không chu trình (Acyclic State-Graph) LangGraph [8] tích hợp Bộ nhớ Đe dọa (Threat Memory) lưu trữ lịch sử xâm nhập để liên kết các hành vi đơn lẻ. Nhằm chống hư cấu (hallucination), hệ thống tích hợp Retrieval-Augmented Generation (RAG) [7] theo mô hình Dual-RAG kết hợp tìm kiếm ngữ nghĩa FAISS [20] và từ khóa BM25 [21], được hợp nhất bằng thuật toán Reciprocal Rank Fusion (RRF) [11]:

$$RRF_Score(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (2.2)$$

truy xuất trực tiếp từ 433 mục STIX MITRE ATT&CK và tài liệu NIST SP 800-61r2 làm căn cứ giải trình pháp y.

2.4 An ninh Đối kháng AI và Mật mã học Kiểm toán Nhật ký

Tích hợp LLM phát sinh nguy cơ bị tấn công tiêm nhiễm câu lệnh qua nhật ký (log-substrate prompt injection) [9], khi kẻ tấn công chen câu lệnh độc hại vào chuỗi User-Agent hoặc tham số URI để thao túng suy luận của AI [30]. Đặc thù của véc-tơ này là **nội dung nhật ký do chính kẻ tấn công soạn ra**: mọi lớp phòng thủ dựa trên nhận dạng nội dung đều là cuộc đua bất tận với biến thể mới. Để triệt tiêu véc-tơ này về mặt cấu trúc, SENTINEL áp dụng cơ chế Đóng gói Dữ liệu Phân định (Delimited Data Encapsulation) với Nonce ngẫu nhiên cô lập dữ liệu thô, kết hợp rào chắn Guardrail tắt định ép đầu ra mô hình nghiêm ngặt trong ba trạng thái (BLOCK_IP, LOG, AWAIT_HITL).

Nhằm bảo vệ vết pháp y, hệ thống áp dụng kỹ thuật HMAC-SHA256 [29] niêm phong từng bản ghi bằng khóa bí mật cục bộ, tạo thành chuỗi kiểm toán chống thao túng sau xâm nhập. Cần phát biểu chính xác phạm vi bảo chứng: vì HMAC dùng **khóa đối xứng dùng chung**, cơ chế này chứng minh được **tính toàn vẹn và khả năng phát hiện giả mạo**, nhưng **không** cấu thành chống chối bỏ theo nghĩa mật mã học—tính chất đó đòi hỏi chữ ký bất đối xứng. Đây là ranh giới lý thuyết được tôn trọng xuyên suốt luận văn.

2.5 Tổng quan Nghiên cứu Liên quan và Khoảng trống Nghiên cứu

Nghiên cứu ứng dụng trí tuệ nhân tạo trong vận hành an ninh mạng đã tiến triển qua ba nhóm giải pháp kiến trúc chính [15].

Nhóm phòng thủ doanh nghiệp truyền thống gồm các nền tảng SIEM như IBM QRadar và giải pháp SOAR như Palo Alto Cortex XSOAR [2, 5]. QRadar tương quan sự kiện bằng tập luật tĩnh nên bộc lộ tỷ lệ báo động giả cao (>80%) do thiếu khả năng hiểu ngữ cảnh; Cortex XSOAR tự động hóa phản hồi bằng playbook nên thất bại khi gặp biến thể ngoài kịch bản. Ưu thế của nhóm này là độ trễ rất thấp—đặc tính mà Tầng 1 của SENTINEL kế thừa thay vì loại bỏ.

Nhóm tác tử AI đám mây tiêu biểu là Watchtower [12] và CyberRAG [13]. Watchtower

gửi nhật ký tới LLM đám mây, kéo theo độ trễ tự chú ý bậc hai $\mathcal{O}(N^2)$ tốn 4–26 giây mỗi sự kiện và vi phạm nguyên tắc Zero-Trust [31]. CyberRAG tập trung truy xuất tri thức đe dọa bằng RAG đơn tầng; thiếu bộ lọc đường truyền nên dễ quá tải, đồng thời thiếu cơ chế có trạng thái để liên kết chuỗi tấn công đa giai đoạn.

Nhóm tác tử tự chủ hoạch định gồm AutoBnB [14], LanG Agentic SOC [16] và Policy-Guided SIEM [17], lần lượt kết hợp mạng Bayes với tác tử hoạch định, mô hình đa tác tử chú trọng quản trị, và rào chắn quy tắc trên Splunk. Các công trình này đạt bước tiến về điều phối đa tác tử, song vẫn vận hành theo kiến trúc đơn tầng—mọi sự kiện đều phải vượt qua nút suy luận LLM; hệ quả là độ trễ trung bình công bố duy trì ở mức 480–520 ms trên hạ tầng 16–24 GB VRAM.

Khoảng trống nghiên cứu được xác định là: **chưa có kiến trúc lai nào giải quyết đồng thời ba yêu cầu**—(1) giải quyết phần lớn lưu lượng ngay tại tầng thống kê/học máy với độ trễ dưới một mili-giây; (2) suy luận nhận thức có trạng thái chạy cục bộ trên GPU 16 GB với rào chắn neo-RAG chống hư cấu; và (3) bảo vệ toàn vẹn mô hình bằng đóng gói phân định Nonce cùng niêm phong vết pháp y HMAC-SHA256. Bảng 1 đối chiếu chi tiết SENTINEL với các nhóm đại diện.

Bảng 1. Bảng so sánh đối chiếu giữa SENTINEL và các giải pháp phòng thủ hiện hành

Tiêu chí Đánh giá	SIEM / SOAR (QRadar, XSOAR)	Tác tử SOTA (Watchtower, LanG, AutoBnB)	Kiến trúc SENTINEL (Luận văn)
Kỹ thuật Cốt lõi	Tập luật tương quan tĩnh, kịch bản Python/JS cứng	Đa tác tử LLM Đám mây / Local Multi-Agent	Bộ lọc Welford $\mathcal{O}(1)$ + LightGBM T1, LangGraph Agent T2
Mục tiêu Vận hành	Gom nhật ký tập trung và phản hồi theo kịch bản	Tự động hóa điều tra sự cố bằng suy luận ngôn ngữ	Giải quyết lưu lượng tại tầng rẻ, dành tầng đắt cho ca cần suy luận
Phân tầng chi phí	Một tầng rẻ, không có suy luận	Một tầng đắt, mọi sự kiện đều qua LLM	Hai tầng: rẻ trước, đắt sau
Chủ quyền Dữ liệu	Cục bộ nội bộ	Rủi ro rò rỉ Cloud hoặc yêu cầu 24GB GPU	Air-gapped 100% trên GPU 16 GB
Chống Prompt Inj.	Không áp dụng	Phụ thuộc bộ lọc xác suất (dễ bị bẻ khóa)	Bảo chứng cấu trúc, độc lập với nội dung payload
Kiểm toán Pháp y	Nhật ký văn bản tiêu chuẩn (dễ bị chỉnh sửa)	Không hỗ trợ niêm phong mật mã	Chuỗi HMAC-SHA256 phát hiện giả mạo

Chương 3

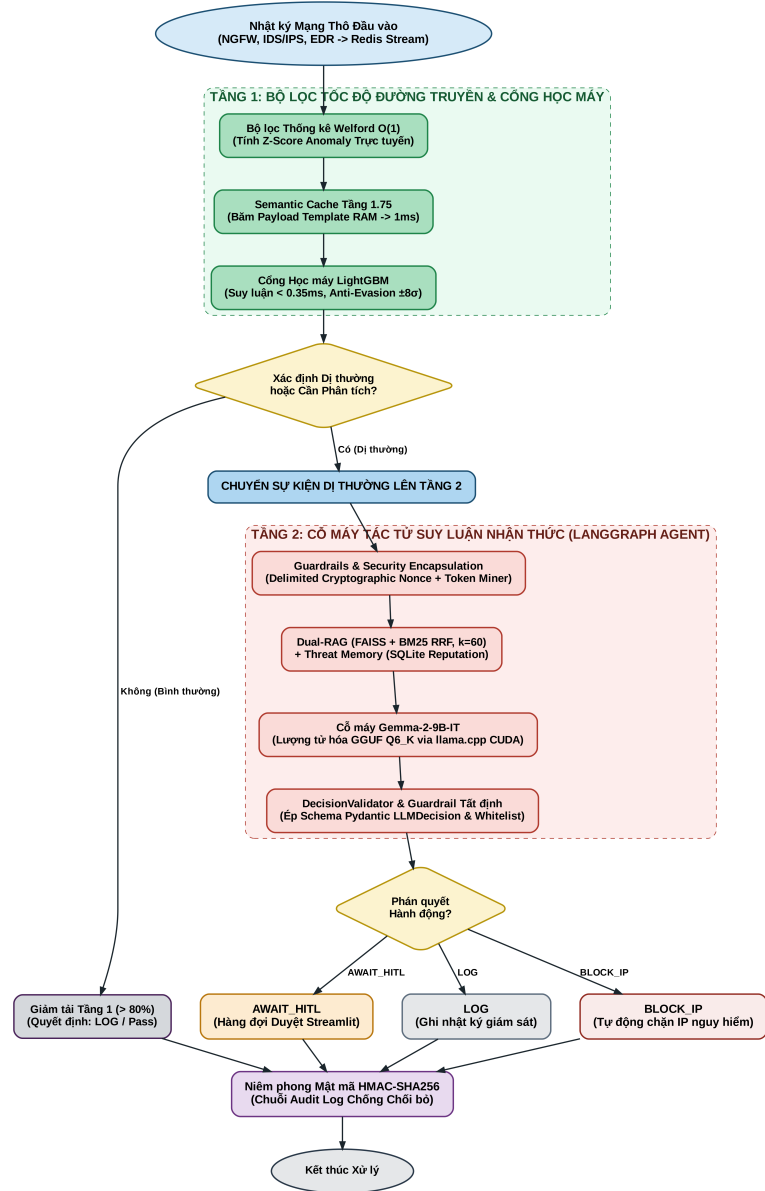
Thiết kế Hệ thống và Triển khai Kỹ thuật

Chương này trình bày thiết kế kiến trúc và giải pháp triển khai của hệ thống SENTINEL: luồng xử lý phân tầng, bộ lọc đường truyền Tầng 1 cùng Cổng Học máy, kiến trúc suy luận tác tử Tầng 2 và Bộ nhớ Đe dọa, cơ chế truy xuất tri thức Dual-RAG, giải pháp an ninh đối kháng cùng kỹ thuật kiểm toán mật mã, và phương án triển khai cục bộ.

3.1 Kiến trúc Tổng thể SENTINEL và Tích hợp Ngăn xếp SOC

SENTINEL áp dụng Kiến trúc Nhận thức Hai tầng với nguyên lý thiết kế cốt lõi là phân tách nhiệm vụ tính toán theo chi phí: Tầng 1 vận hành các thuật toán thống kê và học máy phi trạng thái để giải quyết dứt điểm phần lớn lưu lượng ở tốc độ đường truyền; Tầng 2 vận hành cỗ máy tác tử ngôn ngữ có trạng thái nhằm suy luận ngữ cảnh sâu cho các sự kiện dị thường phức tạp.

Hình 1 minh họa luồng thực thi. Nhật ký thô từ các nguồn giám sát ranh giới (NGFW, IDS/IPS, EDR) được nạp vào hàng đợi Redis Stream. Ngay khi sự kiện tới, Tầng 1 tính Z-score trực tuyến bằng thuật toán Welford và phân loại tương tác đặc trưng qua Cổng Học máy LightGBM. Các luồng tuân theo phân bổ nền hoặc trùng mẫu truy vấn đã lưu trong Semantic Cache được phán quyết dưới mili-giây, không chạm tới GPU.



Hình 1. Luồng thực thi vận hành trong pipeline SENTINEL. (Nguồn: Tác giả)

Một điểm thiết kế quyết định cần nêu rõ: trong danh mục hành động của Tầng 1, **chỉ trạng thái ESCALATE mới dẫn tiếp lên Tầng 2**. Các phán quyết BLOCK_IP, ALERT, AWAIT_HITL và DROP đều **kết thúc tại Tầng 1**. Đây là cơ sở để định nghĩa tỉ lệ xả tải ở Chương 4: một sự kiện được xem là đã gỡ tải khi nó không phát sinh lời gọi mô hình ngôn ngữ, bất kể phán quyết cuối là chặn hay bỏ qua.

Tại Tầng 2, cỗ máy tác tử LangGraph kích hoạt chuỗi suy luận bằng mô hình Foundation-Sec-8B-Instruct cục bộ, kết hợp lịch sử từ Bộ nhớ Đe dọa và tri thức truy xuất từ Dual-RAG. Phán quyết cuối được kiểm soát bởi rào chắn tắt định giới hạn trong ba trạng thái, và mọi

bản ghi đều được niêm phong bằng chuỗi băm HMAC-SHA256. Trong ngăn xếp SOC doanh nghiệp, SENTINEL vì vậy đóng vai trò middleware nhận thức đứng giữa nền tảng thu thập SIEM và công cụ thực thi tường lửa/SOAR: hấp thụ lưu lượng nhiều tại Tầng 1 và cung cấp căn cứ giải trình có trích dẫn cho sự kiện leo thang, đồng thời bảo đảm chủ quyền dữ liệu nhờ vận hành air-gapped hoàn toàn trên GPU đơn 16 GB VRAM.

3.2 Tầng 1: Bộ lọc Tốc độ Đường truyền và Cổng Học máy

Tầng 1 là rào chắn phi trạng thái đầu tiên, kết hợp bộ lọc thống kê Welford $\mathcal{O}(1)$ (Công thức 2.1) với Cổng Học máy LightGBM nhằm bảo đảm độ trễ mili-giây trước khi kích hoạt GPU.

Bộ lọc thống kê tính $Z = (x - \mu)/\sigma$ theo thời gian thực trên các đặc trưng lưu lượng (tần suất gói tin, dung lượng luồng, thời lượng kết nối), với μ và σ cập nhật theo công thức truy hồi ở Mục 2.2 mà không lưu lịch sử thô [6]; lưu lượng tuân theo phân bố nền bị loại ngay lập tức. Bên cạnh nhánh thống kê, Tầng 1 vận hành tập chữ ký ứng dụng gồm **29 họ tấn công web** với chuẩn hoá đa vòng theo lỗi OWASP CRS—giải mã URL và thực thể HTML trước khi khớp mẫu—nhằm bắt payload nguy trang bằng mã hoá. Đối với hành vi lặp lại diện rộng, Semantic Cache Tầng 1.75 băm chuỗi truy vấn đã chuẩn hoá trong RAM và tái sử dụng phán quyết có sẵn.

Cổng Học máy LightGBM tiếp nối, đánh giá các tương tác đặc trưng phi tuyến. Mô hình dùng cây phát triển theo lá, huấn luyện trên 949.535 mẫu NetFlow (664.674 huấn luyện, 189.907 hold-out). Nhằm triệt tiêu né tránh học máy, Cổng ML kẹp Z -score trong $\pm 8\sigma$ và điền giá trị thiếu bằng trung bình baseline; nếu tỷ lệ đặc trưng biến dạng vượt 30%, nó **từ chối phán quyết** và nâng cấp lên Tầng 2. Abstain là lựa chọn có chủ đích: một câu trả lời sai kèm độ tin cậy cao nguy hiểm hơn lời thú nhận không biết, vì nó khiến hệ tự hành động mà không ai kiểm lại. Phán quyết được ánh xạ sang hành động theo **chính sách bốn dải độ tin cậy** (0,85 / 0,65 / 0,40 tương ứng BLOCK_IP / ESCALATE / ALERT / LOG), bảo đảm độ tin cậy *quyết định* hành động thay vì chỉ được ghi kèm, và cô lập phần tự động chặn vào dải cao nhất—nơi còn đòi hỏi sự đồng thuận của luật Tầng 1.

3.3 Tầng 2: Tác tử Nhận thức LangGraph và Bộ nhớ Đe dọa

Tầng 2 đảm nhận suy luận nhận thức có trạng thái, vận hành mô hình Foundation-Sec-8B-Instruct lượng tử hóa GGUF Q4_K_M [23] qua llama.cpp biên dịch cờ CUDA trên GPU cục bộ 16 GB VRAM, cắt VRAM yêu cầu từ 18 GB (FP16) xuống 7–8 GB mà không suy giảm đáng kể độ chính xác.

Quy trình được điều phối bởi đồ thị LangGraph [8] qua năm trạm: lọc rào chắn (guardrails), làm giàu ngữ cảnh (rag_context), phân loại sơ bộ (llm_triage), ánh xạ kỹ thuật (attack_mapper), và thực thi (action_executor hoặc human_in_loop). Trạm ánh xạ đòi hỏi căn cứ xác thực từ kho tri thức; nếu không có mã kỹ thuật phù hợp rõ ràng, đồ thị định tuyến sang kiểm duyệt chuyên gia (AWAIT_HITL) thay vì đoán mò. Đường chuyển-người là **một đầu ra thiết kế, không phải thất bại**: hệ thống *phân loại* việc cần người chứ không thay thế người. Tại llm_triage, hệ thực hiện **gỡ nhãn** (label stripping), loại mọi trường phân loại nội bộ khỏi prompt—điều kiện tiên quyết cho tính hợp lệ của mọi số đo ở Chương 4, vì nhãn thật lọt vào ngữ cảnh sẽ khiến mô hình chép đáp án chứ không suy luận.

Để liên kết hành vi rải rác theo thời gian, Tầng 2 duy trì Bộ nhớ Đe dọa trên SQLite tối ưu chỉ mục, cấu hình synchronous=NORMAL với rollback-journal thay cho WAL nhằm tránh xung đột quyền truy cập cross-UID trong Docker. Bộ nhớ quản lý uy tín IP (ip_reputation), thực thể nội bộ (known_entities), chỉ báo rủi ro (apt_indicators) và sự kiện đe dọa (threat_events). Điểm uy tín S suy giảm theo thời gian im lặng t :

$$S_{\text{new}} = S_{\text{old}} \times (1 - \lambda)^t \quad (3.1)$$

với λ là hệ số suy giảm. Khi đạt ngưỡng tối đa ($S = 100$), Tầng 1 áp dụng chặn ngay lập tức cho kết nối tiếp theo mà không kích hoạt lại Tầng 2—khép kín vòng phản hồi tự động, biến chi phí suy luận một lần thành phán quyết dừng lại được nhiều lần. Các IP hạ tầng nội bộ nằm trong danh sách trắng cứng, miễn nhiễm với lệnh chặn nhằm ngăn tự từ chối dịch vụ.

3.4 Cơ chế Truy xuất Tri thức Lai Dual-RAG

Để hạn chế hư cấu và bảo đảm minh bạch cho quyết định an ninh, Tầng 2 tích hợp Dual-RAG trên kho 433 mục MITRE ATT&CK chuẩn STIX 2.1 [24] cùng tài liệu NIST SP 800-61r2 [10]. Truy xuất chạy song song hai cơ chế hỗ trợ: tìm kiếm ngữ nghĩa bằng vector FAISS [20] với mô hình Sentence-BERT [25], phát hiện các kỹ thuật cùng bản chất nhưng khác từ khóa; và tìm kiếm từ khóa thưa BM25 [21], bảo đảm khớp chính xác các định danh như mã CVE, khóa Registry hoặc số hiệu cổng. Kết quả được hợp nhất bằng RRF (Công thức 2.2) với $k = 60$ rồi chèn vào ngữ cảnh tại `rag_context`.

Trên nền truy xuất này, hệ thống áp dụng **rào chắn neo bằng chứng**: mọi mã kỹ thuật do mô hình công bố đều bị đối chiếu ngược với tập tài liệu đã truy xuất của chính lô đó; mã không hiện diện bị ép về N/A và chuyển `AWAIT_HITL`. Đây là ràng buộc **kiểm chứng được của kiến trúc**, không phụ thuộc vào xu hướng hư cấu của mô hình cụ thể—và chính là ranh giới phân biệt một hệ thống có tri thức với một lớp bọc quanh mô hình ngôn ngữ.

3.5 Kiến trúc An ninh AI và Niêm phong Kiểm toán Mật mã

Tiêm nhiễm câu lệnh qua nhật ký [9] là kỹ thuật chèn câu lệnh điều khiển vào các trường nhật ký thô (User-Agent, URI, nội dung Syslog) nhằm chiếm quyền điều khiển hành vi mô hình. SENTINEL áp dụng Đóng gói Dữ liệu Phân định với nonce mật mã sinh mới cho mỗi lô xử lý [30]: toàn bộ nhật ký thô được bọc trong cặp thẻ chứa nonce, buộc mô hình xử lý nội dung bên trong như văn bản thuần túy. Sinh nonce theo từng lô—thay vì dùng chuỗi phân định cố định—là bắt buộc, vì dấu phân định bất biến có thể bị suy ra từ mã nguồn hoặc quan sát lặp lại, khi đó kẻ tấn công có thể tự đóng vùng bọc để thoát ra. Song song, rào chắn tắt định ép phán quyết tuân thủ cấu trúc Pydantic (`LLMDecision`); khi định dạng lỗi, cơ chế phục hồi bằng Regex trích xuất trường cốt lõi hoặc chuyển về trạng thái an toàn `AWAIT_HITL`.

Về kiểm toán, mỗi bản ghi phán quyết thứ i được liên kết chuỗi bằng HMAC-SHA256 [29]:

$$H_i = \text{HMAC-SHA256}(D_i \parallel H_{i-1}, K) \quad (3.2)$$

trong đó D_i là dữ liệu bản ghi hiện tại, H_{i-1} là băm bản ghi liền trước, và K là khóa bí mật nạp từ biến môi trường—không mã hoá cứng trong nguồn, vì khóa lộ trong mã khiến toàn chuỗi có thể bị làm giả lại từ đầu. Liên kết chuỗi khiến mọi sửa đổi hay xóa bản ghi giữa chuỗi làm đứt gãy tính liên tục và bị phát hiện tức thì. Như đã nêu ở Mục 2.4, cơ chế này bảo đảm **toàn vẹn và phát hiện giả mạo**; do dùng khóa đối xứng, nó không cấu thành chống chối bỏ theo nghĩa mật mã học.

3.6 SOC Dashboard và Kiến trúc Triển khai Cục bộ

Bảng điều khiển SOC Analyst Dashboard được xây dựng trên Streamlit, kết nối trực tiếp Redis Stream và Bộ nhớ Đe dọa để cung cấp góc nhìn thời gian thực: phễu xả tải theo từng tầng, dòng thời gian chuỗi tấn công đa giai đoạn, chỉ báo toàn vẹn chuỗi HMAC, và bảng kiểm duyệt HITL. Toàn bộ dịch vụ đóng gói bằng Docker Compose; `llama.cpp` biên dịch cờ CUDA cho card RTX 4060 Ti; `threat_memory.db` và chỉ mục FAISS gắn trên Docker Volume độc lập để điểm uy tín IP và vết pháp y sống sót qua tái khởi động.

Thiết kế tuân thủ chuẩn Zero Trust [31]: chuỗi mật khẩu trong `REDIS_URL` được che phủ trước khi ghi log. Nhằm chịu tải khi lưu lượng tăng đột biến, hệ thống áp dụng backpressure dựa trên **độ tồn đọng (lag) của Redis consumer group** thay vì độ dài hàng đợi. Lựa chọn này mang tính bắt buộc: độ dài một Redis Stream chỉ tăng theo thời gian kể cả khi mọi bản ghi đã xử lý xong, nên dùng nó làm tín hiệu điều tiết sẽ khiến hệ tự bóp nghẹt nguồn phát vĩnh viễn; chỉ tồn đọng của consumer group mới phản ánh đúng phần công việc còn lại.

Chương 4

Thực nghiệm và Đánh giá

Chương này trình bày kết quả thực nghiệm định lượng đối với hệ thống SENTINEL, tổ chức theo đúng ba câu hỏi nghiên cứu đặt ra ở Chương 1. Mỗi mục nêu chỉ số đo được, mẫu số của phép đo, và giới hạn của kết luận rút ra từ chỉ số đó.

4.1 Môi trường Thực nghiệm và Giao ước Đo lường

Hệ thống được triển khai trên máy chủ cục bộ đại diện cho SOC doanh nghiệp vừa và nhỏ, toàn bộ vi dịch vụ ảo hóa bằng Docker Compose trong môi trường air-gapped. Nguyên tắc phương pháp luận xuyên suốt là **mẫu số phải khớp câu hỏi**: chỉ số phụ thuộc phân bố lưu lượng (tỉ lệ xả tải) đo trên luồng thực, chất lượng phân loại đo trên tập cân bằng lớp. Trộn hai loại này là nguồn sai lệch nghiêm trọng—đo tỉ lệ xả tải trên tập ép cân bằng 50/50 sẽ cho kết quả thấp giả tạo, vì tập đó không phản ánh phân bố mà hệ thống thực sự gặp. Bảng 2 nêu rõ vai trò từng tập.

Hai giao ước bổ sung áp dụng cho mọi lượt đo. Thứ nhất, luật động bị **đóng băng** trong suốt phép đo, tránh việc phép đo tự sinh luật rồi tự hưởng lợi ở vòng sau. Thứ hai, 50 mẫu do tác giả biên soạn lẫn trong `ground_truth.json` bị loại khỏi mọi tỉ lệ—nếu giữ lại, chúng chiếm 16,7% tập chấm quy kết và toàn bộ cùng một đáp án, làm lệch phân bố nhãn.

Bảng 2. Cấu hình Thử nghiệm và Phân vai Tập Dữ liệu

Thành phần	Thông số	Vai trò trong phép đo
Vi xử lý (CPU)	Intel Core i7	Bộ lọc Welford và luồng Redis
Bộ nhớ RAM	32 GB DDR5	Semantic Cache và chỉ mục FAISS
Card đồ họa (GPU)	RTX 4060 Ti 16 GB	llama.cpp CUDA, Foundation-Sec-8B
Ảo hóa	Docker Compose	Môi trường air-gapped, tái lập được
datatest.json	4.240 mẫu, 51,8% tấn công	Chất lượng phân loại Cổng ML. Không dùng cho tỉ lệ xả tải
ground_truth.json	1.750 mẫu → 1.700 khi chấm	Ablation, chất lượng lập luận. 50 mẫu tác giả biên soạn bị loại khỏi mọi tỉ lệ
build_stream()	25.799 (150 khởi động + 25.649 chấm), 31,6% tấn công	Tỉ lệ xả tải, toàn tuyến
Luồng dạng SOC	99.717 sự kiện, 9,8% tấn công	Tỉ lệ xả tải ở base-rate gần thực tế
Tập đối kháng	823 mẫu, trong đó 703 từ nguồn công bố	AdvBench, deepset, jackhhao. Phần biên soạn báo tách riêng

4.2 RQ1 — Xả tải và Kinh tế học Tầng lọc

RQ1 gồm ba vế: xả tải ở tốc độ đường truyền, cắt độ trễ, và giảm tiêu thụ tài nguyên. Như đã nêu ở Mục 3.1, một sự kiện được tính là **đã gỡ tải** khi nó không phát sinh lời gọi mô hình ngôn ngữ—mọi hành động khác ESCALATE đều kết thúc tại Tầng 1. Định nghĩa này quan trọng vì cách đếm chỉ tính DROP sẽ bỏ sót BLOCK_IP, ALERT và AWAIT_HITL—vốn cũng không tiêu tốn một token nào.

Bảng 3. Tỉ lệ Xả tải theo Từng Tầng trên Hai Luồng có Base-rate Khác nhau

Tầng giải quyết	Luồng benchmark 25.649 sk		Luồng dạng SOC 99.717 sk	
	Số sự kiện	Tỉ lệ	Số sự kiện	Tỉ lệ
Tầng 1 (luật + Welford + chữ ký)	20.663	80,6%	38.207	38,3%
Cổng Học máy (LightGBM 4 dải)	2.568	10,0%	58.992	59,2%
Cộng dồn — không chạm LLM	23.231	90,6%	97.199	97,5%
Còn lại → Tầng 2 (LLM)	2.418	9,4%	2.518	2,5%
Tỉ lệ tấn công của luồng		31,6%	9,8%	

Kết quả ở Bảng 3 cho thấy một đặc tính chưa được phát biểu tường minh trong các công

trình liên quan: **tỉ lệ xả tải là hàm của tỉ lệ tấn công trong luồng, không phải hằng số của hệ thống**. Trong phạm vi một luồng, tỉ lệ tổng tách tuyến tính thành xả tải(p) = $(1 - p) \cdot \text{xả tải}_{\text{lãnh}} + p \cdot \text{xả tải}_{\text{tấn công}}$ với p là tỉ lệ tấn công. Trên luồng benchmark, hai tỉ lệ theo nhân là **92,35%** với ca lãnh và **86,72%** với ca tấn công, tái lập đúng con số 90,6% đo được; trên luồng dạng SOC, hai tỉ lệ là **97,88%** và **93,77%**, tái lập đúng 97,5%. Bản thân hai hệ số khác nhau giữa hai luồng vì thành phần lưu lượng của chúng khác nhau, nên quan hệ này cho phép chiếu sang tỉ lệ tấn công khác **trong cùng một luồng**—chứ không cho phép mang hệ số của luồng này áp sang luồng kia. Vì vậy con số 90,6% trên luồng benchmark **không phải giới hạn kiến trúc** mà là hệ quả của việc luồng thử cố ý nhồi tấn công gấp hơn ba lần thực tế.

Một quan sát thứ hai: **tầng gánh chính đổi theo luồng**. Ở luồng benchmark, luật Tầng 1 giải quyết 80,6%; ở luồng lãnh áp đảo, Cổng ML mới là tầng gánh chính với 59,2%. Hai tầng bù trừ cho nhau chứ không chồng lấn—căn cứ thực nghiệm cho việc phải có cả hai.

Tiết kiệm theo từng thành phần. Cổng ML xử lý mỗi véc-tơ trong **0,287 ms**, đạt **3.452 sự kiện/giây**. Bộ đệm ngữ nghĩa Tầng 1.75 đạt tỉ lệ trúng **81,3%** (1.220/1.500 truy vấn) với hệ số tăng tốc **8,9×** và không có lần trục xuất nào; khóa đệm là băm của chuỗi truy vấn đã chuẩn hoá—khớp chính xác chứ không phải tương đồng embedding—nên tỉ lệ trúng đến từ việc nhiều sự kiện khác nhau quy về cùng một mô tả dịch vụ/cổng/kỹ thuật. Một chỉ số chất lượng thuộc về đúng mục này chứ không phải mục bên cạnh, vì nó là điều kiện cạnh của chính tuyên bố xả tải: tỉ lệ 90,6% cũng đọc được theo nghĩa “cho qua gần hết” nếu phần được xả tải không được phán quyết đúng. Trên tập cân bằng `datatest.json`, Cổng ML đạt **MCC 0,667** và Balanced Accuracy 0,833—chủ ý dùng MCC thay Accuracy, vì trên luồng lệch lớp một hệ chỉ cần đoán “lãnh” cho mọi sự kiện đã đạt Accuracy 74% mà không phân biệt được gì—còn phần **tự động chặn không cần người** gồm 962 lệnh đạt **Precision 100%, không báo động giả nào**. Điều kiện sau là bắt buộc: xả tải mà chặn nhằm khách thật thì không phải hiệu quả mà là tổn thất.

Vòng phản hồi Tầng 2 → Tầng 1. Cơ chế xả tải thứ tư được kiểm chứng bằng thực nghiệm hai vòng trên cùng hạt giống. Sau khi thu 598 luật từ phán quyết chặn của Cổng ML, tỉ lệ

leo thang giảm từ **20,28% xuống 17,98%** (giảm tương đối **11,35%**, tức 594 sự kiện được Tầng 1 hấp thụ thêm), trong khi chất lượng phát hiện **tăng** chứ không giảm: MCC +0,0378, Recall +0,0452. Điểm này đáng chú ý vì cơ chế học từ phán quyết thường đánh đổi độ chính xác lấy tốc độ; ở đây cả hai cùng cải thiện.

Độ trễ toàn tuyến. Độ trễ được đo trên 500 sự kiện lấy mẫu bước nhảy từ `build_stream()`, giữ nguyên tỉ lệ lành/tấn công thật thay vì ép cân bằng. Độ trễ trung bình toàn tuyến giảm từ **17.187,9 ms** (LLM đơn tầng) xuống **5.286,7 ms**, tức giảm **69,2%**, và chênh lệch có ý nghĩa thống kê theo kiểm định Mann-Whitney U ($p \approx 9 \times 10^{-57}$). Trung vị còn ấn tượng hơn nhiều: **0,88 ms** so với **17.174,7 ms**, tức khoảng **19.500 lần**—bởi sự kiện điển hình không bao giờ chạm tới LLM. Độ trễ trung bình theo từng chặng xác nhận đúng chênh lệch chi phí là động cơ của kiến trúc: **0,182 ms** tại Tầng 1, **0,924 ms** tại Cổng ML, và **22.785 ms** khi phải gọi LLM.

Một kết quả phải báo cáo dù bất lợi: **phân vị 95 của hệ hai tầng xấu hơn—25.829 ms** so với **21.434 ms** của cấu hình chỉ LLM. Nguyên nhân mang tính cấu trúc: sự kiện nào leo thang thì đã trả chi phí Tầng 1 và Cổng ML rồi mới trả tiếp chi phí LLM. Phần đuôi vì vậy thực sự chậm hơn, và lợi ích của kiến trúc tập trung ở trung vị chứ không trải đều. Báo trung bình và trung vị mà giấu phần đuôi là trình bày sai bản chất đánh đổi. Về tài nguyên, luận văn báo cáo **số lỗi gọi LLM tránh được** (97.199 trên 99.717 sự kiện) thay vì tỉ lệ VRAM giải phóng, vì không có phép đo VRAM nào được thực hiện.

Đối chứng với các cấu hình khác. Để trả lời câu hỏi “nhanh hơn cái gì”, bốn cấu hình đối chứng được đo trên **cùng một mẫu con 150 sự kiện phân tầng** (Bảng 4), thay vì chép số headline từ công trình khác—phép so đó không hợp lệ vì khác tập dữ liệu và khác định nghĩa chỉ số.

Bảng 4. Thông lượng các cấu hình đối chứng trên cùng mẫu con 150 sự kiện phân tầng

Cấu hình đối chứng	Thông lượng
H1 — Chữ ký tĩnh, 29 họ WAF	11.129 sk/s
SENTINEL — Tầng 1 + Cổng ML	3.860 sk/s
H2 — LightGBM đơn tầng, ngưỡng phẳng	2.162 sk/s
H3 — LLM đơn tầng (cùng dữ liệu, mô hình và prompt)	0,19 sk/s

Đường tắt định của SENTINEL chạy nhanh hơn **trên 20.000 lần** so với việc đẩy mọi sự kiện lên mô hình ngôn ngữ (đo 20.531 lần). Chữ ký tính còn nhanh hơn nữa, khoảng ba lần SENTINEL, và điều đó đúng như dự kiến—chúng không thực hiện phép suy luận thống kê hay mô hình nào. Chính thứ tự này *là* luận điểm kiến trúc gói trong một dòng: mỗi tầng tồn đúng phần chi phí của nó, và câu hỏi thiết kế là giải quyết được bao nhiêu lưu lượng trước khi chạm tầng đất. Hai trong số các con số trên không được cộng gộp: mức 0,19 sk/s đẩy từng sự kiện qua **toàn bộ tác tử Tầng 2** (797,9 s cho 150 sự kiện), còn mốc 17.187,9 ms là **lời gọi mô hình trần** trên log thô, không giới hạn token và không lược đồ đầu ra. Hai con số mô tả hai thiết kế đơn tầng khác nhau, không phải một phép đo quy sang hai đơn vị. Cuối cùng, phép thử tải dồn trên luồng 99.717 sự kiện xác nhận cơ chế điều tiết ngược dựa trên độ trễ nhóm tiêu thụ giữ mức sử dụng RAM và VRAM ổn định, bảo đảm tính sẵn sàng liên tục.

4.3 RQ2 — Rào chắn An ninh AI và Toàn vẹn Pháp y

Mô hình đe dọa và giao ước đo. RQ2 nêu tiềm ẩn qua nền nhật ký là rủi ro đối kháng *điển hình*, không phải toàn bộ phạm vi. Vì vậy SENTINEL được đánh giá trước **hai mặt tấn công khác hẳn nhau về cấu trúc**. Mặt thứ nhất tấn công **qua ngôn ngữ**: văn bản tiềm ẩn và bẻ khoá nhúng trong các trường nhật ký, nhắm vào mô hình ngôn ngữ ở Tầng 2; kháng được nghĩa là chỉ thị nhúng không làm đổi hành động của tác tử. Mặt thứ hai tấn công **qua không gian đặc trưng**: bơm giá trị cực đoan hoặc vô cực vào thuộc tính luồng để ép Cổng ML LightGBM trả lời “lạnh”; kháng được nghĩa là mẫu đã bị bóp méo vẫn không bị lật thành lạnh. Một phòng tuyến chỉ phủ mặt thứ nhất là một phòng tuyến hẹp, vì hai tầng bị tấn công bằng hai cách hoàn toàn khác nhau.

Ba giao ước làm cho các tỉ lệ đọc được. Thứ nhất, phán quyết của lớp tính là phép HOẶC của **ba bộ dò tách rời**—khớp mẫu, vô hiệu hoá mã hoá, và bóc ký tự phân định—mỗi bộ ghi nhận riêng, để khi hỏng thì *định vị* được chứ không chỉ đếm được. Thứ hai, **đối chứng âm là bắt buộc**: cùng ba bộ dò đó chạy trên 230 nhật ký lạnh, bởi một lớp gán cờ mọi thứ cũng đạt “chặn 100%”, và một tỉ lệ chặn nêu mà thiếu tỉ lệ báo nhầm là con số không diễn

giải được. Thứ ba, theo đúng Mục 4.1, mẫu từ nguồn công bố và mẫu do tác giả biên soạn được **chấm riêng, không bao giờ gộp vào một con số headline**.

Bảng 5. Các lớp rào chắn, mẫu số phép đo và kết quả

Lớp rào chắn	Mẫu số	Kết quả
<i>Tầng 1 — tấn công qua ngôn ngữ</i>		
Lớp lọc tĩnh, nguồn công bố	703 mẫu	132 → 18,8%
theo bộ dò: khớp mẫu / mã hoá / phân định	703	114 / 29 / 0
Lớp lọc tĩnh, mẫu tác giả biên soạn	80 mẫu	58 → 72,5%
Lớp lọc tĩnh, tập kiểm chéo	40 mẫu	2 → 5,0%
Lớp lọc tĩnh — báo nhảm trên nhật ký lành	230 nhật ký lành	46 → 20,0%
trong đó: lớp nhận dạng mã hoá	46/46	toàn bộ
<i>Tầng 1 — tấn công qua không gian đặc trưng</i>		
Cổng ML — né tránh, chế độ khó (bóp méo mọi đặc trưng; 13 ca bị lật, CI95 97,86–99,27)	1.036 ca tấn công đã bắt	1.023 → 98,75%
Cổng ML — chế độ một đặc trưng (∞ / cực đoan)	1.036 × 2	100% cả hai
Cổng ML — từ chối phán quyết khi đầu vào biến dạng	4.236 sự kiện	116 → leo thang
<i>Tầng 2 — bảo chứng cấu trúc</i>		
Đóng gói dữ liệu phân định bằng nonce	678 mẫu khó, độ phủ 100%	678 → 100%
Kiểm lược đồ khi ép hồng cổ máy suy luận	1 lần ép hồng	AWAIT_HITL, không sụp hành động trùng khớp 0 mẫu đối phán quyết xấu nhất 1.655
Tắt định (cùng hạt giống, 5 lượt)	5 lượt	
Đổi riêng hạt giống	10 mẫu	
Chống tràn ngữ cảnh	cửa sổ 16.384 token, ngân sách 4.000	
<i>Toàn vẹn pháp y</i>		
HMAC — ba kiểu giả mạo cốt lõi	30 lượt mỗi kiểu	100%
HMAC — đối chứng âm (chuỗi nguyên vẹn)	450 bản ghi	không báo động giả
HMAC — cắt đuôi chuỗi	—	0% — không phát hiện được

Tầng 1 trước tấn công qua ngôn ngữ. Trên nguồn công bố, lớp lọc tĩnh chặn **18,8%**, và con số đó phải đọc kèm tỉ lệ báo nhảm **20,0%** trên nhật ký lành. Phân tích theo từng bộ dò định vị vấn đề chính xác: toàn bộ 46 ca báo nhảm đến từ lớp nhận dạng mã hoá, còn lớp khớp mẫu và lớp bóc phân định không sinh ca nào—một chỉ dẫn kỹ thuật về một lớp cụ thể, không phải bản án cho cả rào chắn. Cái giá của một ca báo nhảm cũng có giới hạn và phải nêu ra, vì con số 20% nghe rất đáng ngại nếu thiếu nó: cờ của lớp tĩnh nâng mức cô lập và chú thích vào bản ghi, nhưng **không** loại bỏ sự kiện và không chặn nguồn. Vì vậy đây là tỉ lệ chú thích thừa, không phải tỉ lệ lưu lượng lành bị từ chối phục vụ. Phần mẫu tác giả biên soạn chặn tới 72,5%, cao hơn hẳn số của nguồn công bố, và khoảng cách đó được giải thích chứ không được bào chữa: 45 trong 80 mẫu ấy là ca vượt mã hoá, mà lớp vô hiệu

hoá mã hoá được dựng ra đúng để trị chúng. Trích con số gộp 23,3% sẽ tăng bốc hệ thống bằng cách đếm cả những mẫu được viết ra khi đã biết trước cơ chế phòng thủ; **18,8% mới là con số luận văn đứng sau.**

Tầng 1 trước tấn công qua không gian đặc trưng. Cổng ML kẹp Z -score trong $\pm 8\sigma$, điền giá trị thiếu bằng trung bình baseline, và từ chối phán quyết khi tỉ lệ đặc trưng biến dạng vượt 30%. Ở **chế độ khó**, nơi *toàn bộ* đặc trưng của một ca tấn công đã bị bắt đúng bị đẩy tới cực trị, **98,75%** số mẫu vẫn kháng được (1.023/1.036; CI95 97,86–99,27), với 13 ca bị lật thành lành. Hai chế độ một-đặc-trưng đạt 100%, và luận văn trích chế độ khó chứ không trích trung bình ba chế độ, vì gộp thêm hai chế độ bảo hoà một cách tầm thường sẽ thổi con số vượt quá mức mà phòng tuyến thực sự chứng minh được. Việc từ chối phán quyết được tính là kết quả của rào chắn chứ không phải thất bại: 116 sự kiện được đẩy lên thay vì đoán bừa, bởi một câu trả lời sai kèm độ tin cậy cao sẽ khiến hệ tự hành động mà không ai kiểm lại.

Tầng 2 và khác biệt về bản chất. Lớp tính là bộ lọc rẻ đặt phía trước; bảo chứng an ninh đến từ **Đóng gói Dữ liệu Phân định**, vốn mang tính cấu trúc—nội dung nằm trong vùng bọc nonce được xử lý như văn bản thụ động *bất kể nó nói gì*. Cơ chế này được đo trên **toàn bộ 678 mẫu thuộc bảy nhóm mà lớp tính không hấp thụ được**, ở **độ phủ 100%** của tập đó. Đây là siêu tập của 543 mẫu thật sự lọt qua lớp tính trong các nhóm ấy, nên phép thử chặt hơn mức cần chứ không phải chọn mẫu dễ; Tầng 2 kháng được **678/678**. Khác biệt về bản chất nằm ở đây: một bộ phân loại chặn $x\%$ luôn để lọt $(100 - x)\%$ và suy giảm trước biến thể mới, trong khi bảo chứng cấu trúc không phụ thuộc vào việc nhận dạng được nội dung độc hại hay không. Con số 18,8% vì vậy **không mâu thuẫn** với luận điểm—nó định lượng chính xác khối lượng công việc mà lớp cấu trúc phải gánh. Hai phép kiểm độ ổn định củng cố mọi số đo phụ thuộc LLM trong chương này: với cùng một hạt giống chạy năm lượt, tác tử đưa ra **hành động trùng khớp hoàn toàn**, và đổi riêng hạt giống trên mười mẫu **không làm đổi phán quyết nào**, nên chênh lệch giữa các cấu hình không thể bị quy cho nhiễu của bộ giải mã. Khi ép cỗ máy suy luận hỏng, hệ suy biến về `AWAIT_HITL` thay vì sụp đổ hoặc rơi về hành động dễ dãi.

Phát hiện giả mạo bằng cách nào. Phép kiểm tính lại toàn chuỗi từ bản ghi gốc theo $H_i = \text{HMAC}(D_i \parallel H_{i-1}, K)$. Vì mỗi mắt xích ăn vào hash của mắt trước, một byte bị đổi ở D_j làm hỏng *mọi* mắt xích từ j trở đi, nên việc phát hiện có **định vị** chứ không chỉ báo “có gì đó đã đổi”. Ba kiểu giả mạo cốt lõi ánh xạ đúng vào ba cách phá tính liên tục: **sửa nội dung** làm hash tính lại tại j không khớp hash đã lưu; **chèn bản ghi giả** tạo ra một mắt xích không có tiền nhiệm hợp lệ để trở về; **xoá bản ghi giữa** khiến H_{j+1} trở tới một hash không còn tồn tại. Cả ba đều bị phát hiện 30/30 lượt, và đối chứng âm trên 450 bản ghi nguyên vẹn không sinh báo động giả nào. Phép đo chạy với khoá nạp từ biến môi trường, không phải khoá mặc định trong mã nguồn—một khoá lộ trong mã cho phép dựng lại cả chuỗi từ đầu và làm mọi tỉ lệ ở trên mất giá trị. Luận văn chủ động nêu điểm mù: **cắt cụt đuôi chuỗi không thể phát hiện được**, đây là tính chất nguyên lý của log-chaining, vì xoá các bản ghi cuối không để lại mắt xích nào phía sau để đối chiếu. Cách khắc phục là neo giá trị hash ra ngoài hệ thống theo chu kỳ.

Những rào chắn không quy về một tỉ lệ. Nhiều cơ chế bảo vệ các đường mà không tỉ lệ chặn nào đo được. RAG_Sanitizer bảo vệ kho tri thức ở cả hai thời điểm—lúc nạp thì chuẩn hoá NFKC chống homoglyph, xoá ký tự điều khiển và ký tự rộng-không, lược thể HTML/JS cùng ảnh và liên kết Markdown; lúc truy xuất thì vô hiệu hoá các chỉ thị mệnh lệnh nhúng trong tài liệu. Thiết kế của nó mang một bài học đo được: một bản trước đó khớp cụm bị chặn ở bất kỳ vị trí nào trong câu, nên đã viết đề lên văn xuôi an ninh hợp lệ, vô hiệu hoá nhằm bốn mục kỹ thuật di chuyển ngang (T1090, T1021, T1021.001, T1553.001) mà chính các kịch bản APT của luận văn cần tới. Việc giới hạn khớp vào **vị trí mệnh lệnh** đã triệt tiêu hiện tượng này, và phép kiểm lại trên kho tri thức 433 mục hiện tại cho **không có** ca vô hiệu hoá nhằm nào—chính là kỷ luật “chặn phải đọc kèm báo nhằm” áp cho một lớp không công bố tỉ lệ nào. FeedbackValidator đóng lại lỗ hổng mà vòng phản hồi Tầng 2 về Tầng 1 ở Mục 4.2 mở ra: luật được thăng cấp từ phán quyết phải qua danh sách trường cho phép, phép kiểm mẫu và chặn dải điểm, vì một cơ chế học từ phán quyết cũng là một cơ chế đầu đọc luật Tầng 1. DecisionValidator áp lược đồ đầu ra, cứu hồi trường sai định dạng bằng biểu thức chính quy, và đòi **Tầng 1 đồng thuận** trước mọi lệnh tự động chặn. Một bộ dò vòng lặp giới hạn số lần thăm mỗi nút ở

mười lượt để tác tử không quẩn vòng đốt thời gian GPU, và một bộ làm sạch đầu ra bóc các khối base64 và thập lục phân trước khi bản ghi tới cơ sở dữ liệu hay bảng điều khiển, khoá đường phát tán ngược qua giao diện chuyên gia.

Vị trí trường, đo tách riêng. Vì cơ chế đóng gói nonce bọc **theo từng trường**, vị trí trường quyết định payload nằm trong hay ngoài vùng bọc—một tính chất mà không con số gộp nào kiểm được, bởi các nhóm còn lại đều đặt payload vào cùng một trường. Tập `field_injection` gồm 100 mẫu công bố rải payload qua bốn vị trí: URI 24, User-Agent 23, message 30, payload 23. Lớp tính chỉ chặn 12/100, đúng như dự kiến với tiềm nhiễu bằng ngôn ngữ đời thường không mang mã hoá và không mang ký tự phân định. Cho chạy qua Tầng 2 ở **độ phủ 100%**, cả 100 mẫu đều kháng được, và kết quả **đồng đều trên mọi vị trí**—24/24, 23/23, 30/30, 23/23—với phán quyết phân bố 73 BLOCK_IP và 27 ALERT, không ca nào tụt về hành động dễ dãi. Chính sự đồng đều mới là phát hiện, chứ không phải con số tổng: nếu cơ chế bọc chỉ áp cho một số trường mà bỏ sót trường khác, các vị trí không được bọc sẽ lộ ra ở đây thành một lỗ hổng. Kết quả này báo tách khỏi con số 678, vì nó trả lời một câu hỏi khác.

4.4 RQ3 — Suy luận Có trạng thái và Quy kết Kỹ thuật

RQ3 gồm bốn vế: quy kết kỹ thuật ATT&CK, truy xuất tri thức, báo cáo minh bạch, và giảm tải người.

Chất lượng lập luận. Phán quyết của tác tử được chấm bởi **trọng tài khác họ mô hình** (Meta-Llama-3-8B-Instruct chấm Foundation-Sec-8B-Instruct) trên thang 1–5, theo đúng khuyến nghị nhằm loại bỏ Thiên kiến Tự đề cao [26]. Trên 277 sự kiện leo thang lên Tầng 2, kết quả bốn trục là: độ chính xác ngữ cảnh **2,68**; độ liên quan câu trả lời **4,22**; độ trung thành với ngữ cảnh **3,88**; độ bao phủ ngữ cảnh **4,13**; trung bình **3,73/5,0**. Chỉ số neo bằng chứng—tỉ lệ lập luận chứa ít nhất một cặp *trường = giá trị* khớp giá trị thật trong log—đạt **50,2%** (KTC 95%: 44,3–56,0), trung bình 1,7 trích dẫn mỗi phán quyết.

Ba nhận định phải nêu kèm. Thứ nhất, trục **độ chính xác ngữ cảnh 2,68** là **điểm yếu rõ**

ràng: tài liệu truy xuất thường chứa mục không liên quan, tức tầng truy xuất chứ không phải tầng sinh mới là nút thắt. Thứ hai, lượt đo này có **69/277 phán quyết thiếu trường bắt buộc**, và chính script tự gắn cờ cảnh báo—các con số trên vì vậy là chỉ dấu định hướng, không phải kết luận đã chốt. Thứ ba, luận văn **không** công bố tỉ lệ khớp chính xác mã ATT&CK trong đợt này, vì các tệp kết quả quy kết hiện có được sinh trước khi áp dụng giao ước loại mẫu biên soạn và mâu thuẫn lẫn nhau; trích dẫn chúng sẽ vi phạm chính nguyên tắc đo lường đặt ra ở Mục 4.1. Mọi tỉ lệ quy kết dù sao cũng phải đọc so với sàn chứ không so với 0: trên tập chấm 250 mẫu thật, kỹ thuật T1595.003 chiếm 130 mẫu, nên một hệ chỉ cần đoán luôn mã đó cho mọi ca đã đạt **52%**.

Truy xuất tri thức và rào chắn neo. Kho tri thức gồm 433 mục MITRE ATT&CK chuẩn STIX, truy xuất bằng RRF hợp nhất FAISS và BM25. Rào chắn neo bằng chứng (Mục 3.4) buộc mọi mã kỹ thuật công bố phải hiện diện trong tài liệu truy xuất của chính lô đó; vi phạm bị ép về N/A và chuyển AWAIT_HITL. Giá trị học thuật của cơ chế nằm ở chỗ nó **không phụ thuộc vào xu hướng hư cấu của mô hình cụ thể**: đổi mô hình khác thì ràng buộc vẫn giữ nguyên hiệu lực.

Phân công kiến trúc giữa hai tầng. Một kết quả tưởng như tiêu cực lại xác nhận thiết kế: trên nhóm mẫu ngoài NetFlow (tấn công web CSIC, zero-day, DAPT), Cổng ML chỉ đạt Recall **11,1%**, và 305 trong 336 ca trượt của toàn tập nằm gọn trong nhóm này. Nguyên nhân mang tính kiến trúc chứ không phải lỗi—Cổng ML học trên **đặc trưng luồng**, trong khi tấn công web nằm ở **payload**, vốn là phần việc của Tầng 2 và tập chữ ký. Hệ quả phương pháp luận là Recall tổng 0,755 đang trộn hai quần thể không cộng được và phải báo tách bạch. Liên quan tới điều đó, như đã nêu ở Mục 3.3, tỉ lệ chuyển chuyên gia **không** phải chỉ số cần tối thiểu hoá: một kiến trúc tuyên bố tỉ lệ chuyển-người bằng 0 trên mọi tình huống là kiến trúc đã bỏ mất rào chắn cuối cùng.

Chương 5

Kết luận và Hướng phát triển

Chương này tổng kết kết quả nghiên cứu và đóng góp của luận văn, khẳng định giá trị khoa học lẫn thực tiễn, phân tích các giới hạn kiến trúc và phác thảo lộ trình phát triển.

5.1 Tổng hợp Kết quả Nghiên cứu

Về **RQ1**, các phép đo xác nhận kiến trúc phân tầng giải quyết được vấn đề tiêu thụ tài nguyên của mô hình ngôn ngữ. Trên luồng benchmark 25.649 sự kiện với 31,6% tấn công, Tầng 1 và Cổng Học máy giải quyết **90,6%** lưu lượng mà không phát sinh lời gọi LLM nào; trên luồng 99.717 sự kiện ở tỉ lệ tấn công gần thực tế hơn (9,8%), con số này đạt **97,5%**. Nghiên cứu chỉ ra tỉ lệ xả tải là **hàm của tỉ lệ tấn công trong luồng** chứ không phải hằng số của hệ: trong mỗi luồng, tỉ lệ tổng tách tuyến tính thành một tỉ lệ cho ca lạnh và một tỉ lệ cho ca tấn công—92,35% và 86,72% trên luồng benchmark, 97,88% và 93,77% trên luồng dạng SOC—nên quan hệ này cho phép chiếu sang tỉ lệ tấn công khác trong cùng một luồng, chứ không cho phép mang hệ số của luồng này áp sang luồng kia. Cổng ML xử lý mỗi véc-tơ trong 0,287 ms (3.452 sự kiện/giây), bộ đệm ngữ nghĩa đạt tỉ lệ trúng 81,3% với hệ số tăng tốc 8,9×, và vòng phản hồi từ Tầng 2 về Tầng 1 giảm thêm 11,35% tải leo thang *đồng thời* cải thiện chất lượng phát hiện (MCC +0,0378). Toàn tuyến, độ trễ trung bình giảm từ **17.187,9 ms** xuống **5.286,7 ms** (−69,2%, Mann-Whitney $p \approx 9 \times 10^{-57}$) và trung vị từ 17.174,7 ms xuống **0,88 ms**. Luận văn báo cáo ngang bằng kết quả ngược chiều: phân vị 95 **xấu hơn** (25.829 ms so với 21.434 ms), vì sự kiện nào leo thang thì đã trả chi phí Tầng 1 và Cổng ML trước khi trả chi phí LLM. Lợi ích tập trung ở trung vị chứ không trải đều.

Về **RQ2**, chuỗi niêm phong HMAC-SHA256 phát hiện **100%** ba kiểu giả mạo cốt lõi—sửa nội dung, chèn bản ghi giả và xoá bản ghi giữa—trên 30 lượt thử mỗi kiểu, với đối chứng âm trên 450 bản ghi nguyên vẹn không sinh báo động giả. Phạm vi bảo chứng được phát biểu chính xác: vì dùng khóa đối xứng, cơ chế bảo đảm **toàn vẹn và phát hiện giả mạo**, không phải chống chối bỏ theo nghĩa mật mã học; và **cắt cụt đuôi chuỗi là điểm mù nguyên lý**. Hệ thống được đánh giá trước hai mặt tấn công khác biệt. Trước tấn công **qua ngôn ngữ**, lớp lọc tĩnh chặn 18,8% trên 703 mẫu từ nguồn công bố—kèm tỉ lệ báo nhầm 20,0% trên log lạnh mà toàn bộ tập trung ở lớp nhận dạng mã hoá—trong khi Tầng 2, đo ở **độ phủ 100%** của 678 mẫu thuộc các nhóm mà lớp tĩnh không hấp thụ được, kháng được **678/678**. Trước tấn công **qua không gian đặc trưng**, nơi giá trị cực đoan được bơm vào để ép Cổng ML nói “lạnh”, tỉ lệ kháng đạt **98,75%** ở chế độ khó bóp méo toàn bộ đặc trưng (1.023/1.036; CI95 97,86–99,27). Kết quả này củng cố luận điểm trung tâm: bảo chứng an ninh đến từ **Đóng gói Dữ liệu Phân định**, vốn xử lý nội dung trong vùng bọc như văn bản thụ động bất kể nó nói gì, nên không phụ thuộc vào năng lực nhận dạng nội dung độc hại của bất kỳ bộ phân loại nào. Phép kiểm độ ổn định xác nhận tác tử tắt định với cùng seed và không đổi phán quyết nào khi chỉ đổi seed, nên chênh lệch đo được giữa các cấu hình không phải nhiễu của bộ giải mã.

Về **RQ3**, tác tử LangGraph Tầng 2 kết hợp Dual-RAG trên 433 mục ATT&CK chuẩn STIX, được trọng tài khác họ mô hình (Meta-Llama-3-8B) chấm trên 277 sự kiện leo thang, đạt trung bình **3,73/5,0** với độ liên quan 4,22 và độ trung thành 3,88, trong khi **độ chính xác ngữ cảnh 2,68 là điểm yếu rõ ràng** chỉ ra nút thất nằm ở tầng truy xuất; tỉ lệ neo bằng chứng đạt 50,2%. Đóng góp kiến trúc ở RQ3 là **rào chắn neo-RAG tắt định**: mọi mã kỹ thuật công bố buộc phải hiện diện trong tài liệu truy xuất của chính lô đó, nếu không sẽ bị ép về N/A và chuyển chuyên gia—biến việc chống hư cấu từ kỳ vọng đặt vào mô hình thành ràng buộc kiểm chứng được, giữ nguyên hiệu lực khi đổi mô hình. Luận văn **không công bố** tỉ lệ khớp chính xác mã ATT&CK ở đợt này, vì các tệp kết quả hiện có được sinh trước giao ước loại mẫu biên soạn và mâu thuẫn lẫn nhau.

5.2 Đóng góp Khoa học và Thực tiễn

Về **đóng góp khoa học**, luận văn đề xuất và định lượng nguyên lý *đặt phán quyết rẽ trước phán quyết đất*, giải quyết rào cản độ trễ bậc hai $\mathcal{O}(N^2)$ của mô hình ngôn ngữ trên luồng dữ liệu liên tục [18]. Đóng góp phương pháp luận đi kèm là việc phát biểu tường minh rằng **tỉ lệ xả tải là hàm của base-rate**, kèm quan hệ tuyến tính cho phép chuyển kết quả sang môi trường có tỉ lệ tấn công khác—điều mà cách báo cáo một con số duy nhất không làm được. Thứ hai, luận văn đóng góp mô hình cô lập dữ liệu bằng Nonce mật mã, cung cấp bảo chứng an ninh **mang tính cấu trúc**, độc lập với các bộ phân loại xác suất vốn suy giảm trước biến thể mới [9, 30]. Thứ ba, luận văn xác lập bộ giao ước đo lường cho hệ tác tử an ninh cục bộ: mẫu số khớp câu hỏi, thước đo miễn nhiễm lệch lớp, loại trừ mẫu tự biên soạn khỏi tỉ lệ công bố, và bắt buộc báo tỉ lệ chặn kèm tỉ lệ báo nhầm.

Về **đóng góp thực tiễn**, luận văn cung cấp nguyên mẫu hoàn chỉnh đóng gói bằng Docker Compose, kiểm chứng trên phần cứng thương mại phổ thông (Intel Core i7, 32 GB RAM, một card RTX 4060 Ti 16 GB VRAM), tuân thủ nguyên tắc Zero-Trust trong môi trường air-gapped [31]. Hệ thống tích hợp bảng điều khiển Streamlit kết nối hàng đợi Con-người-trong-vòng-lặp nhằm thay thế kịch bản SOAR tĩnh [5], kèm niêm phong vết pháp y bằng chuỗi băm HMAC-SHA256 [29].

5.3 Giới hạn và Hướng Phát triển

Bốn giới hạn dưới đây được nêu kèm giải pháp đối ứng tương ứng.

Thứ nhất—phạm vi nguồn nhật ký. Đánh giá hiện tập trung vào đặc trưng NetFlow chuẩn hóa và nhật ký HTTP Tầng 7 [1, 27]. Các nguồn phi cấu trúc (Windows Event Log, Syslog thô, AWS CloudTrail) đòi hỏi cơ chế phân tích cú pháp phức tạp hơn. Hướng đối ứng là xây dựng cỗ máy trích xuất tích hợp thuật toán phân nhóm biểu mẫu log như Drain [22] để chuẩn hóa mọi định dạng về véc-tơ đặc trưng trước khi vào đường ống.

Thứ hai—ranh giới chịu tải trên một nút. Khi lưu lượng vượt 100.000 sự kiện/giây, lượng

sự kiện leo thang lên Tầng 2 trên GPU đơn 16 GB có thể vượt năng lực suy luận tức thời, gây tích tụ hàng đợi. Hướng đối ứng là chuyển bộ lọc Tầng 1 sang ngôn ngữ biên dịch (Go hoặc Rust), di trú Redis Stream sang Apache Kafka phân mảnh ghi đĩa, và nâng cấp Tầng 2 lên vLLM hoặc TGI hỗ trợ PagedAttention và continuous batching.

Thứ ba—quy kết kỹ thuật và chất lượng truy xuất. Trục độ chính xác ngữ cảnh 2,68 cho thấy tầng truy xuất đưa vào nhiều tài liệu không liên quan; đồng thời lượt đo chất lượng lập luận có 69/277 phán quyết thiếu trường bắt buộc nên chỉ mang tính định hướng. Hướng đối ứng gồm: đo lại quy kết theo giao ước đã chuẩn hoá kèm sán đoán bừa 52%, bổ sung chỉ số truy xuất ($\text{recall}@k$, MRR) để tách lỗi truy xuất khỏi lỗi sinh, và kiểm định độ tin cậy của trọng tài LLM bằng hệ số Cohen's κ đối chiếu với người chấm.

Thứ tư—tương quan chuỗi tấn công và tự từ chối dịch vụ. Cơ chế liên kết hành vi hiện dựa trên địa chỉ IP nguồn cố định, nên trong môi trường xoay IP hoặc botnet phân tán, hiệu quả sẽ giảm. Đồng thời, để tránh chặn nhầm hạ tầng nội bộ, hệ thống buộc các phán quyết dựa trên quy tắc chung chung về hàng đợi chuyên gia, chấp nhận giảm mức tự động hóa. Hướng đối ứng là phát triển tương quan dựa trên **véc-tơ kỹ thuật** thay vì địa chỉ nguồn, kết hợp Mạng Thần kinh Đồ thị, và áp dụng Học tăng cường từ Phản hồi của Con người trên các quyết định phê duyệt tại hàng đợi `AWAIT_HITL` để dần tự động hóa an toàn.

Ngoài bốn hướng trên, lộ trình dài hạn bao gồm nâng chuỗi niêm phong lên chữ ký bất đối xứng (Ed25519) nhằm đạt tính chống chối bỏ thực sự, neo chu kỳ giá trị băm ra ngoài hệ thống để bịt điểm mù cắt đuôi, và tích hợp chuẩn Model Context Protocol cho phép tác tử điều phối động các công cụ an ninh mở rộng.

Khép lại một nỗ lực nghiên cứu nghiêm túc, nhưng mở ra những khát vọng sâu xa trong hành trình bảo vệ không gian số bằng sức mạnh của Trí tuệ Nhân tạo Tác tử.

Tài liệu tham khảo

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [2] G. González-Granadillo, S. González-Zarzosa, and R. Diaz, "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures," *Sensors*, vol. 21, no. 14, art. 4759, 2021.
- [3] R. Zuech, T. M. Khoshgoftaar, and R. Wald, "Intrusion Detection and Big Heterogeneous Data: A Survey," *Journal of Big Data*, vol. 2, art. 3, 2015.
- [4] S. Tariq, M. Baruwat Chhetri, S. Nepal, and C. Paris, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys*, vol. 57, no. 9, art. 224, 2025.
- [5] Ismail, R. Kurnia, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, "Toward Robust Security Orchestration and Automated Response in Security Operations Centers with a Hyper-Automation Approach Using Agentic Artificial Intelligence," *Information*, vol. 16, no. 5, art. 365, 2025.
- [6] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] LangChain AI, "LangGraph: Multi-Actor, Stateful LLM Applications," *GitHub repository*, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [9] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, "More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models," *arXiv preprint arXiv:2302.12173*, 2023.
- [10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Revision 2, 2012.
- [11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Bootstrap Product Algorithms," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.
- [12] R. Pandey and A. Bhujang, "Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content," *arXiv preprint arXiv:2605.24421*, 2026.
- [13] F. Blefari, C. Cosentino, F. A. Pironti, A. Furfaro, and F. Marozzo, "CyberRAG: An Agentic RAG cyber attack classification and reporting tool," *Future Generation Computer Systems*, vol. 176, art. 108186, 2026.
- [14] Z. Liu and A. Anwar, "AutoBnB-RAG: Enhancing Multi-Agent Incident Response with Retrieval-Augmented Generation," *arXiv preprint arXiv:2508.13118*, 2025.
- [15] J. Zhang *et al.*, "When LLMs Meet Cybersecurity: A Systematic Literature Review,"

- Cybersecurity*, vol. 8, 2025.
- [16] A. Abdennebi, N. Kara, L. Lahlou, and H. Ould-Slimane, “LanG – A Governance-Aware Agentic AI Platform for Unified Security Operations,” *arXiv preprint arXiv:2604.05440*, 2026.
 - [17] R. Sahay *et al.*, “Policy-Guided Threat Hunting: An LLM Enabled Framework with Splunk SOC Triage,” *arXiv preprint arXiv:2603.23966*, 2026.
 - [18] A. Vaswani *et al.*, “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
 - [19] A. Gholami *et al.*, “A Survey of Quantization Methods for Efficient Neural Network Inference,” *arXiv preprint arXiv:2103.13630*, 2021.
 - [20] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
 - [21] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–380, 2009.
 - [22] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *Proceedings of the IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.
 - [23] Cybersecurity Research Team, “Foundation-Sec: An Open LLM for Cyber Threat Detection,” *Security Preprint*, 2024.
 - [24] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, Technical Report MP180360, 2018.
 - [25] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
 - [26] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
 - [27] S. Myneni, A. Chowdhary, A. Sabur, S. Sengupta, G. Agrawal, D. Huang, and M. Kang, “DAPT 2020 – Constructing a Benchmark Dataset for Advanced Persistent Threats,” in *Deployable Machine Learning for Security Defense (MLHat 2020)*, Communications in Computer and Information Science, vol. 1271, Springer, 2020, pp. 138–163.
 - [28] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv:2309.15217*, 2023.
 - [29] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” Internet Engineering Task Force, RFC 2104, 1997.
 - [30] OWASP GenAI Security Project, “OWASP Top 10 for Large Language Model Applications,” Version 2025, The OWASP Foundation, 2025. [Online]. Available: <https://genai.owasp.org/>
 - [31] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
 - [32] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and Transferable Adversarial Attacks on Aligned Language Models,” *arXiv preprint arXiv:2307.15043*, 2023.
 - [33] J. Hao *et al.*, “Jailbreak Prompts on Large Language Models,” *GitHub repository*, 2023. [Online]. Available: https://github.com/verazuo/jailbreak_llms

- [34] Deepset, “Prompt Injections Dataset,” *Hugging Face repository*, 2023. [Online]. Available: <https://huggingface.co/datasets/deepset/prompt-injections>